Universidade Federal de Minas Gerais
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC642 – Introdução à Inteligência Artificial (2026/2)

Trabalho Prático 2 – Redes Neurais Artificiais

Data de entrega: 22/11/2026, 23:59h

1 Introdução

Neste trabalho, você irá implementar uma **rede neural artificial** (RNA) de duas camadas (uma escondida e uma de saída) para a classificação de dígitos escritos à mão. A RNA será treinada no conjunto de dados **MNIST**, que possui 70.000 imagens com resolução 28×28 dos dígitos de 0 a 9.

O objetivo principal é praticar a modelagem e a implementação de RNAs, incluindo a **propagação das entradas**, as **funções de ativação**, a **função de perda** e o **gradiente descendente**, bem como analisar o impacto dos hiperparâmetros no desempenho da rede. Além de codificar a RNA, você deverá redigir um **relatório** descrevendo sua metodologia e analisando os resultados experimentais obtidos.

2 Código-base

2.1 Executar o servidor

O código-base do trabalho está **anexado à tarefa no Moodle**. Ele inclui um cliente em JavaScript para configurar os hiperparâmetros da RNA, iniciar o seu treinamento e salvar e carregar resultados (pesos e curvas de aprendizado), além de um servidor em Python, onde a RNA é implementada e treinada. Para executar o servidor, instale as dependências e inicie-o com os seguintes comandos:

Terminal

```
python -m venv tp2-env          # Criar ambiente virtual Python
source tp2-env/bin/activate     # Ativar ambiente virtual
pip install -r requirements.txt
python server.py -port 5001     # Executar o servidor na porta 5001
```

Assim que o servidor iniciar, abra no navegador o endereço `http://localhost:5001`. Essa URL deve mostrar a interface de desenho e visualização da RNA, como na Figura 1.

2.2 Arquivos e ponto de entrada

Todo o seu trabalho será realizado no arquivo `learn.py`. Antes de começar a sua implementação, abra esse arquivo e familiarize-se com as funções definidas. Em seguida, estude o arquivo `server.py` para entender como ele chama as funções de `learn.py` durante o treinamento da rede.

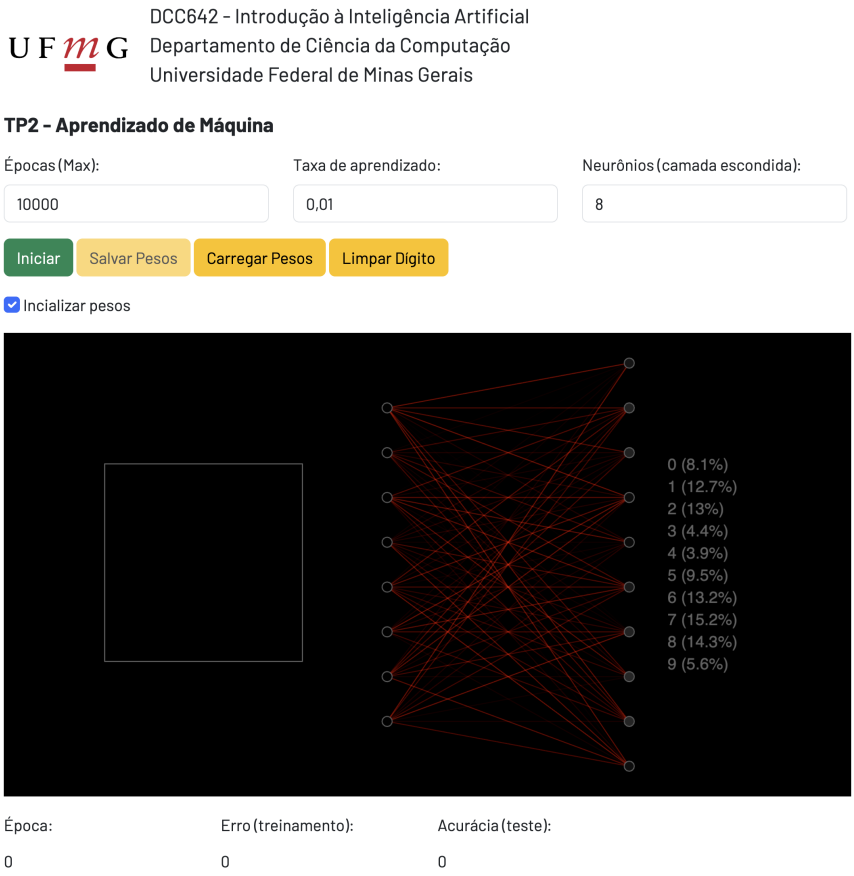


Figura 1: Interface de desenho e visualização da RNA.

3 Tarefas

3.1 Inicialização dos parâmetros (pesos)

Implemente a função `initialize_parameters(input_size, hidden_size, output_size)` para inicializar os parâmetros (pesos) da RNA. Os argumentos representam o número de neurônios nas camadas de entrada, escondida e de saída, respectivamente:

- **Entrada:** como as imagens têm resolução 28×28, `input_size` $= 28 \times 28 = 784$;
- **Escondida:** `hidden_size` é um hiperparâmetro configurado no cliente, no campo *Neurônios (camada escondida)*;
- **Saída:** como o MNIST tem dígitos de 0 a 9, temos um problema de classificação multiclasse com $C = 10$ classes, logo `output_size` $= 10$.

A função deve retornar uma tupla com quatro arranjos NumPy, `(W1, b1, W2, b2)`, onde `W1` e `b1` são os pesos da primeira camada e `W2` e `b2` os da segunda. Lembre-se de que os pesos W devem ser inicializados com valores aleatórios próximos de zero, enquanto os pesos b podem ser inicializados com zero. Portanto:

- utilize `np.random.randn` multiplicado por `0.01` para inicializar `W1` (`input_size` × `hidden_size`) e `W2` (`hidden_size` × `output_size`);
- utilize `np.zeros` para inicializar `b1` ($1 \times$ `hidden_size`) e `b2` ($1 \times$ `output_size`).

Resultados esperados

```
W1, b1, W2, b2 = learn.initialize_parameters(784, 4, 10)
----
W1: (784, 4)
[[ 0.00496714 -0.00138264  0.00647689  0.0152303 ]
 [-0.00234153 -0.00234137  0.01579213  0.00767435]
 [-0.00469474  0.0054256  -0.00463418 -0.0046573 ]
 ...
 [-0.00139397  0.00739653 -0.01909356  0.01318302]
 [ 0.00072681 -0.00411493 -0.00089234 -0.00037571]
 [-0.01731201  0.01494938  0.00041326  0.00443002]]

b1: (1, 4)
[[0. 0. 0. 0.]]

W2: (4, 10)
[[ 0.0095155  -0.01021162  0.00473472 -0.00267641  0.00846771 -0.02127227
  -0.00099094 -0.00602821  0.00432263  0.00470044]
 [-0.00707626 -0.00712174 -0.00110665 -0.00896642  0.00841984 -0.00369207
  -0.02906988 -0.00374822 -0.01038544 -0.01631276]
 [-0.01237427  0.00109408  0.01328641  0.00313184 -0.00606503  0.00455904
  -0.0045909  -0.006946   -0.01154363 -0.01751829]
 [-0.00389924  0.00158053 -0.00096624 -0.00415967 -0.00945746  0.00608246
  -0.01317132  0.00776028 -0.01002217 -0.00752435]]

b2: (1, 10)
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

3.2 Funções de ativação

Antes de implementar a propagação das entradas, você precisa implementar as funções de ativação da RNA. Neste trabalho, usaremos a ReLU como ativação da camada escondida e a softmax na camada de saída, pois temos um problema multiclasse.

ReLU. Implemente a função `relu(z)`, aplicada elemento a elemento, como visto em sala de aula:

$$\text{relu}(z) = \max(0, z)$$

Softmax. Implemente a função `softmax(z)`, aplicada a cada linha (exemplo) de `z`, como visto em sala de aula:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{C} e^{z_k}}, \qquad j = 1, \ldots, C$$

Resultados esperados

```
a = learn.relu(np.array([[-1, 0, 1], [-2, 0, 2]]))
----
a: [[0 0 1]
    [0 0 2]]

a = learn.softmax(np.array([[1, 2, 3], [4, 5, 6]]))
----
a: [[0.09003057 0.24472847 0.66524096]
    [0.09003057 0.24472847 0.66524096]]
```

3.3 Propagação das entradas

Implemente a função `forward(X, parameters)` para propagar as entradas `X` pela RNA definida por `parameters` (tupla `(W1, b1, W2, b2)`). Para facilitar o cálculo dos gradientes, calcule a ativação de cada camada em duas etapas: (i) combinação linear e (ii) função de ativação:

$$Z_1 = XW_1 + b_1, \qquad A_1 = \text{relu}(Z_1),$$
$$Z_2 = A_1W_2 + b_2, \qquad A_2 = \text{softmax}(Z_2).$$

Note que a entrada da camada de saída é a ativação da camada escondida A_1, e não a matriz de exemplos X. A função deve retornar a tupla `(Z1, A1, Z2, A2)`. Utilize a função `np.dot` para as multiplicações de matrizes.

Resultados esperados

```
X = np.random.randn(2, 784)
Z1, A1, Z2, A2 = learn.forward(X, parameters)
----
Z1: (2, 4)
[[ 0.24242511 -0.1806328  -0.32022768  0.05051037]
 [-0.3853982  -0.13327416  0.02946697  0.13882065]]

A1: (2, 4)
[[0.24242511 0.         0.         0.05051037]
 [0.         0.         0.02946697 0.13882065]]

Z2: (2, 10)
[[ 0.00210984 -0.00239572  0.00109901 -0.00085894  0.00157509 -0.00484971
  -0.00090552 -0.00106941  0.00054169  0.00075945]
 [-0.00090593  0.00025165  0.00025738 -0.00048516 -0.00149161  0.00097871
  -0.00196373  0.00087261 -0.00173144 -0.00156075]]

A2: (2, 10)
[[0.10025104 0.09980037 0.10014976 0.09995386 0.10019745 0.09955576
  0.09994921 0.09993283 0.10009396 0.10011576]
 [0.09996714 0.10008293 0.1000835  0.10000921 0.09990861 0.10015572
  0.09986145 0.10014509 0.09988465 0.0999017 ]]
```

3.4 Função de perda

Implemente a função `cross_entropy_loss(y_true, y_pred)` para calcular a entropia cruzada entre os rótulos reais `y_true` (codificados em *one-hot*) e as previsões da RNA `y_pred`. Como visto em sala de aula, para m exemplos e C classes, a perda é definida como:

$$L = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=1}^{C} y_{ic}\log\hat{y}_{ic}$$

Utilize a função `np.log` para o logaritmo e `np.sum` para o somatório.

Resultados esperados

```
y_true = np.array([[0, 1, 0], [1, 0, 0]])
y_pred = np.array([[0.1, 0.6, 0.3], [0.3, 0.4, 0.3]])
loss = learn.cross_entropy_loss(y_true, y_pred)
----
loss: 0.8573992140459634
```

3.5 Gradiente descendente

Implemente a função `gradient_descent_step(X, y, parameters, learning_rate)` para executar uma única época do gradiente descendente:

1. utilize a função `forward()` para propagar as entradas `X` pela RNA;
2. calcule o erro l da previsão atual com a função `cross_entropy_loss()`;
3. calcule os gradientes com a função `backward()` (já implementada);
4. atualize os pesos `W1`, `b1`, `W2` e `b2` com a regra de atualização do gradiente descendente: $w \leftarrow w - \alpha \frac{\partial L}{\partial w}$, onde α é a taxa de aprendizado.

A função deve retornar, nessa ordem, a tupla com os pesos atualizados `(W1, b1, W2, b2)` e o erro atual l da rede.

Resultados esperados

```
X = np.random.randn(2, 784)
y = np.array([[0, 1, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]])
parameters = (W1, b1, W2, b2)
parameters, loss = learn.gradient_descent_step(X, y, parameters, learning_rate=0.01)
----
loss: 2.3007908193979407

W1: (784, 4)
[[ 0.00489277 -0.0013802   0.00647982  0.01522034]
 [-0.00232832 -0.00234474  0.01579393  0.00766823]
 [-0.00470117  0.00545288 -0.00465534 -0.00458549]
 ...
 [-0.00136562  0.00739478 -0.01909403  0.01318462]
 [ 0.00067495 -0.00411447 -0.00088931 -0.00038599]
 [-0.01731262  0.01497452  0.0003934   0.00449739]]

b1: (1, 4)
[[-4.63539605e-05  2.45369176e-05 -1.63779507e-05  5.55761532e-05]]

W2: (4, 10)
[[ 0.00950049 -0.01007663  0.0047197  -0.00269141  0.00845266 -0.02128728
  -0.00100588 -0.00604321  0.00430764  0.00468546]
 [-0.00718544 -0.0062819  -0.00121602 -0.00907558  0.00831039 -0.00380133
  -0.0291786  -0.00371489 -0.01049451 -0.01642172]
 [-0.01248018  0.00098765  0.01317978  0.0030256  -0.00617091  0.00445233
  -0.00469654 -0.00599127 -0.0116493  -0.01762391]
 [-0.00409178  0.00138703 -0.00116009 -0.00435282 -0.00964994  0.00588846
  -0.01336337  0.00949599 -0.01021428 -0.00771637]]

b2: (1, 10)
[[-0.00099922  0.00399859 -0.0010031  -0.00100043 -0.0010005  -0.00100286
  -0.00099571  0.00399782 -0.00099769 -0.0009969 ]]
```

3.6 Avaliação do modelo

Como este é um problema de classificação, você avaliará a sua rede pela **acurácia**, isto é, a taxa de acerto da rede no conjunto de teste. Implemente a função `accuracy(y_true, y_pred)` para calcular a acurácia entre os rótulos reais `y_true` e as previsões `y_pred`. Essa implementação pode ser feita de maneira vetorizada, em uma única linha, com as funções `np.mean` e `np.argmax`.

Resultados esperados

```
y_true = np.array([[0, 1, 0], [1, 0, 0]])
y_pred = np.array([[0.1, 0.6, 0.3], [0.3, 0.4, 0.3]])
acc = learn.accuracy(y_true, y_pred)
----
acc: 0.5
```

4 Relatório

Após concluir a implementação, você deverá executar experimentos variando os hiperparâmetros da RNA e reportar os resultados encontrados. Para gerar as curvas de aprendizado, execute o script `plot.py`, que produz os gráficos de erro e de acurácia a partir dos arquivos `model/losses.csv` e `model/accuracies.csv`. A Figura 2 mostra exemplos dessas curvas.

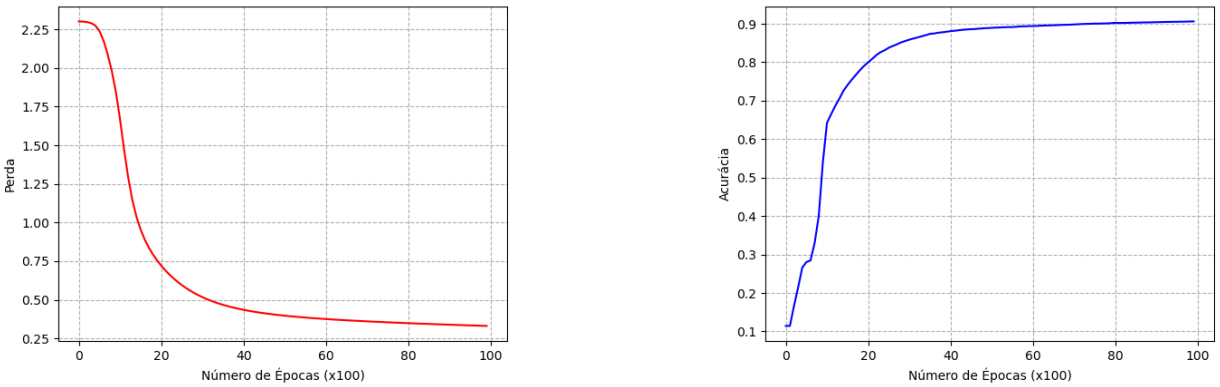


Figura 2: Exemplos de curva de erro (esquerda) e de curva de acurácia (direita).

Em cada experimento, varie apenas o hiperparâmetro indicado e mantenha os demais fixos (informe no relatório quais valores foram usados). Reporte o erro final de treinamento e a acurácia de teste final para as seguintes configurações de hiperparâmetro:

1. **Taxa de aprendizado:** 0,001, 0,01, 0,1 e 0,5
2. **Número de épocas**: 50, 100, 200 e 500
3. **Número de neurônios na camada escondida**: 4, 16, 64 e 256

A partir dos experimentos executados, escolha os hiperparâmetros que maximizem a acurácia de teste da RNA. **A sua acurácia de teste deve ser de no mínimo 90%.**. Apresente as curvas de erro e de acurácia dessa configuração. Desenhe pelo menos 3 dígitos na interface e descreva percepções qualitativas (quais dígitos a rede acerta com facilidade, quais confunde e possíveis motivos).

O seu relatório deve conter **no máximo 3 páginas** no formato de duas colunas da *Association for the Advancement of Artificial Intelligence* (AAAI). O relatório deve conter:

1. **Introdução e Objetivo.**
2. **Metodologia:** arquitetura da RNA, funções de ativação, função de perda, treinamento e decisões de projeto.
3. **Experimentos e Resultados:** tabelas e gráficos com as métricas propostas.
4. **Discussão:** análise crítica dos resultados, *trade-offs* (por exemplo, convergência vs. estabilidade, capacidade da rede vs. custo computacional) e limitações.